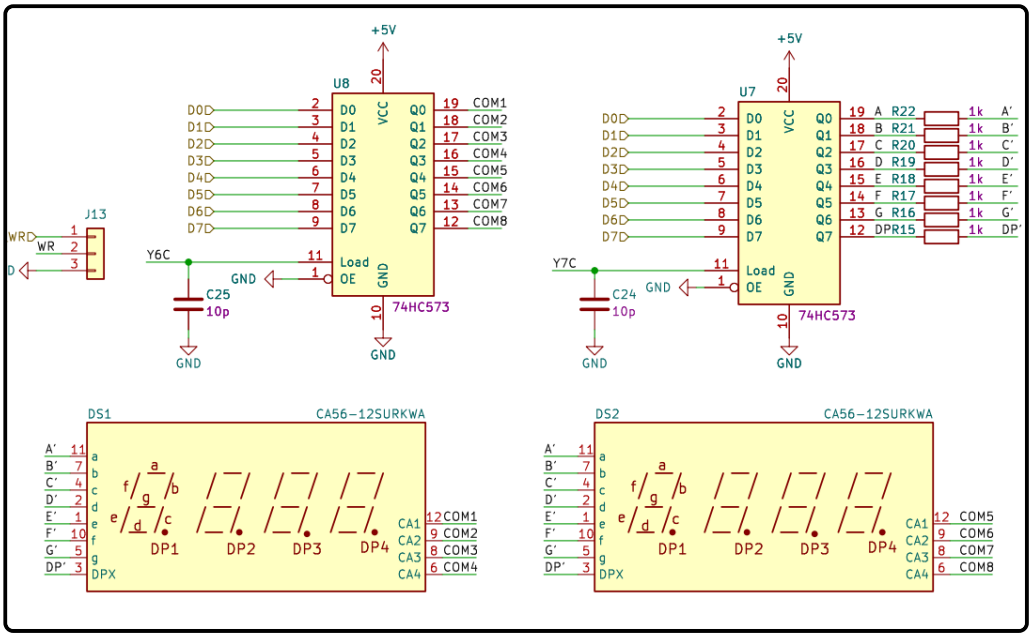
数码管基础代码

数码管工作原理

1. 基本概念

数码管的控制涉及两个核心概念：

- **段选**：控制单个数码管内部哪些段（a,b,c,d,e,f,g,dp）需要点亮
- **位选**：控制多位数码管中的哪一位数码管被激活



2. 位选原理

位选通过移位操作实现，规则如下：

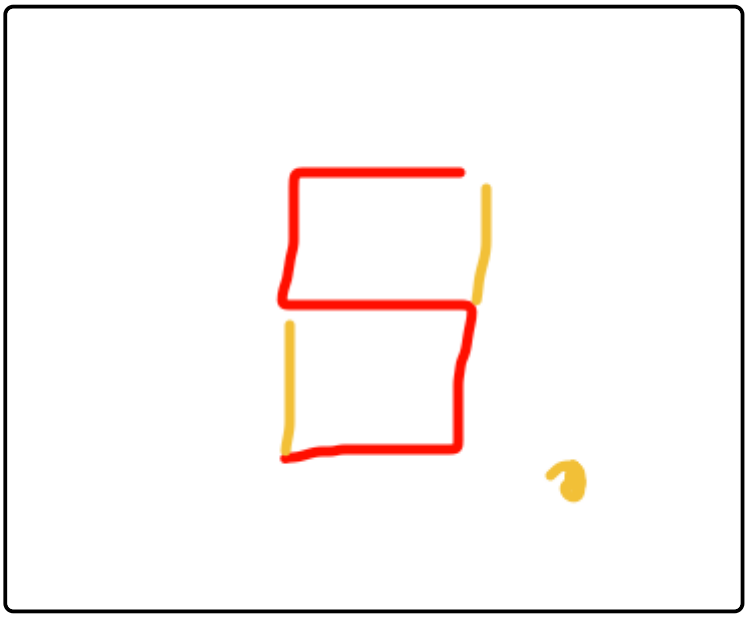
- P0=0000 0001(0x01)：选中第1个数码管
- P0=0000 0010(0x02)：选中第2个数码管
- 实现方式：`P0 = 0x01 << wela`（将1左移wela位）

3. 段选原理

段选采用低电平点亮方式：

- 0：对应段点亮
- 1：对应段熄灭

示例：显示数字"5"



从右下角dp开始，逆序写出二进制：1001 0010 = 0x92

4. 段码表

```
code unsigned char seg_dula[] = {
    0xc0, // 0
    0xf9, // 1
    0xa4, // 2
    0xb0, // 3
    0x99, // 4
    0x92, // 5
    0x82, // 6
    0xf8, // 7
    0x80, // 8
    0x90, // 9
    0xff, // 熄灭
    0x88  // A
};
```

> 注意：虽然官方会提供标准段码表，但务必掌握段码的推导方法。在考试中可能遇到特殊字符（如"-"对应0xbf）需要自行推导。

5. 小数点显示原理

数码管的小数点（dp段）同样采用低电平点亮方式。在段码中，dp位于最低位（二进制最左位）。为了简化小数点的控制，我们采用以下方法：

1. 使用独立的布尔变量（point）控制小数点的显示状态
2. 当需要显示小数点时，通过与运算将段码的最高位置0
   - `P0 &= 0x7f`：0x7f = 0111 1111，将最高位置0，其他位保持不变
3. 这种方式避免了为带小数点的数字单独制作段码表，大大简化了代码实现

示例代码：

```c
P0 = seg_dula[dula];        // 先显示数字对应的段码
if (point) P0 &= 0x7f;      // 如果需要显示小数点，将dp段置0
P2 = P2 & 0x1f | 0xe0;
```

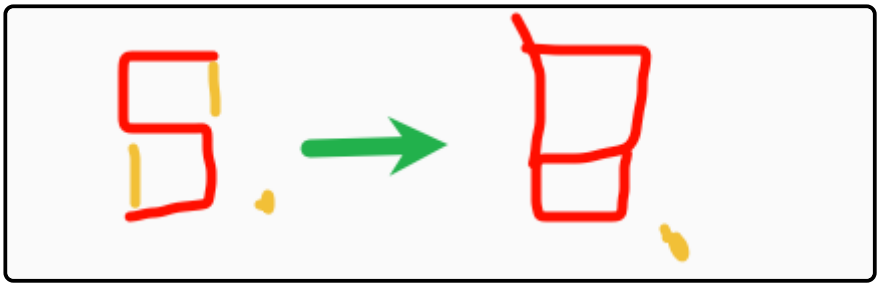
6. 消隐处理

在数码管显示过程中，消隐（Blanking）是一个重要的技术处理环节。它的主要作用是防止数字切换时出现"鬼影"现象。

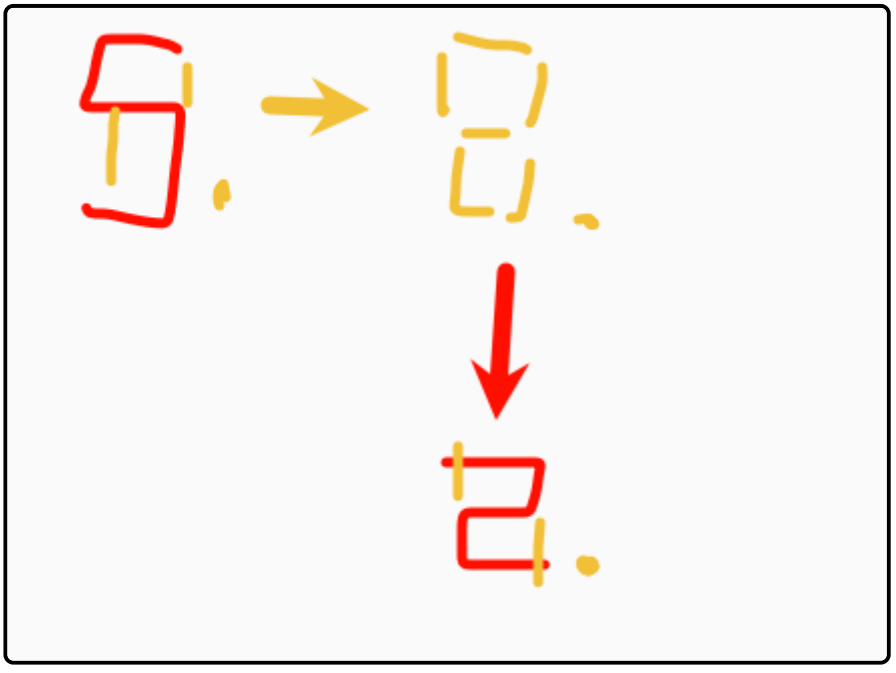
为什么需要消隐？

以显示数字从"5"切换到"2"为例：

1. **不进行消隐：**
   - 当直接从"5"切换到"2"时，由于数码管的余辉效应和人眼视觉暂留特性
   - 两个数字的段码会发生叠加
   - 最终人眼会看到错误的"8"（如下图所示）



2. **进行消隐：**
   - 在切换数字之前，先将所有段置为熄灭状态（P0 = 0xff）
   - 这样可以清除上一个数字的显示状态
   - 确保新数字显示的准确性（如下图所示）



底层代码

在 `Seg_Disp` 函数中，通过以下代码实现消隐：

```
// 消隐处理
P0 = 0xff;          // 所有段置1（熄灭状态）
P2 = P2 & 0x1f | 0xe0;
P2 &= 0x1f;
```

数码管底层函数

```
void Seg_Disp(unsigned char wela, unsigned char dula, unsigned char point) {
    // 消隐
    P0 = 0xff;
    P2 = P2 & 0x1f | 0xe0;
    P2 &= 0x1f;

    // 位选
    P0 = 0x01 << wela;
    P2 = P2 & 0x1f | 0xc0;
    P2 &= 0x1f;

    // 段选
    P0 = seg_dula[dula];
    if (point) P0 &= 0x7f;  // 控制小数点显示
    P2 = P2 & 0x1f | 0xe0;
    P2 &= 0x1f;
}
```

参数说明

- `wela`：位选值，决定点亮哪个数码管
- `dula`：段选值，决定显示什么数字
- `point`：小数点控制，1为显示小数点，0为不显示

主函数代码

数据定义

```
idata unsigned char Seg_Pos;                                    // 数码管显示位置
pdata unsigned char Seg_Buf[8] = {10, 10, 10, 10, 10, 10, 10, 10}; // 数码管显示缓存，初始化为熄灭状态
```

数码管处理函数

```c
void Seg_Proc()
{
    // 数码管处理逻辑可以在这里添加
    Seg_Buf[0] = 1 + ',';  // 显示"1."
}
```

小数点显示优化

为了简化小数点的控制，我们采用了一个巧妙的方法：利用ASCII码的特性来处理小数点的显示。

具体实现原理：

1. 段选数组`seg_dula[]`的有效索引通常不会超过20

2. 当需要显示小数点时，将显示数字加上逗号（','的ASCII码为44）

3. 由于44大于20，可以通过判断缓存值是否大于20来确定是否需要显示小数点

4. 显示时只需将缓存值减去逗号的ASCII码即可得到原始数字

定时器中断服务函数

```c
void Timer1_Isr(void) interrupt 3
{
    uwTick++;                           // 系统计时增加
    Seg_Pos = (++Seg_Pos) % 8;          // 循环更新数码管位置

    // 数码管显示处理：自动判断是否需要显示小数点
    if (Seg_Buf[Seg_Pos] > 20)
        Seg_Disp(Seg_Pos, Seg_Buf[Seg_Pos] - ',', 1);  // 显示带小数点的数字
    else
        Seg_Disp(Seg_Pos, Seg_Buf[Seg_Pos], 0);        // 显示不带小数点的数字
}
```

这种设计方案的优点：

1. 无需额外的小数点控制数组，节省内存空间

2. 代码实现简单，易于理解和维护

3. 显示逻辑集中，减少了出错可能

数码管常用考点

负数显示+高位熄灭

在最近的省赛和国赛中，负数校准值频繁出现，因此我们的显示也需要支持负数。假设我们要显示的范围是-99到99，构建显示逻辑其实很简单，核心是正确显示负号。这里假设负号的段选值为11。

实现方法是先取校准值的相反数，然后根据数值范围判断并分段显示。虽然代码稍长，但能确保目标准确实现。

```c
char Calibration = 0;
void Seg_Proc()
{
  unsigned char Temp_Calibration = -Calibration; // 先手获取一个校准值相反数
  // 代表会占用三个位置显示
  if (Calibration <= -10)
  {
    Seg_Buf[5] = 11; //-
    Seg_Buf[6] = Temp_Calibration / 10 % 10;
    Seg_Buf[7] = Temp_Calibration % 10;
  }
  // 代表占用两个位置显示
  else if (Calibration < 0)
  {
    Seg_Buf[5] = 10;
    Seg_Buf[6] = 11; //-
    Seg_Buf[7] = Temp_Calibration % 10;
  }
  // 代表占用一个位置显示
  else if (Calibration < 10)
  {
    Seg_Buf[5] = 10;
    Seg_Buf[6] = 10;
    Seg_Buf[7] = Calibration % 10;
  }
  // 代表占用两个位置显示
  else
  {
    Seg_Buf[5] = 10;
    Seg_Buf[6] = Calibration / 10 % 10;
    Seg_Buf[7] = Calibration % 10;
  }
}
```

高位熄灭

高位熄灭是数码管显示中的一个重要考点，用于消除数值前导零，提升显示效果。

实现要求：

1. 仅在题目明确要求时实现高位熄灭
2. 每个数码管位置只能赋值一次
3. 禁止使用while循环实现

为什么避免多次赋值？

- 第一次赋值后可能立即触发显示中断
- 随后的赋值可能与中断显示产生冲突
- 最终导致数码管出现闪烁现象

两种实现方案：

1. **数值比较法**
   - 判断显示数值是否大于对应位权值
   - 大于则显示实际数值，否则熄灭（显示10）

- 逻辑简单直观

2.**零值检测法**

- 结合当前位值和前一位状态
- 当前位为0且前一位熄灭时，当前位也熄灭
- 更符合从高到低的处理逻辑

```
    //普通显示
    Seg_Buf[0] = Freq / 10000000 % 10;
    Seg_Buf[1] = Freq / 1000000 % 10;
    Seg_Buf[2] = Freq / 100000 % 10;
    Seg_Buf[3] = Freq / 10000 % 10;
    Seg_Buf[4] = Freq / 1000 % 10;
    Seg_Buf[5] = Freq / 100 % 10;
    Seg_Buf[6] = Freq / 10 % 10;
    Seg_Buf[7] = Freq % 10;
    //高位熄灭1
    Seg_Buf[0] = (Freq >= 10000000)
                    ? Freq / 10000000 % 10
                    : 10;
    Seg_Buf[1] = (Freq >= 1000000)
                    ? Freq / 1000000 % 10
                    : 10;
    Seg_Buf[2] = (Freq >= 100000)
                    ? Freq / 100000 % 10
                    : 10;
    Seg_Buf[3] = (Freq >= 10000)
                    ? Freq / 10000 % 10
                    : 10;
    Seg_Buf[4] = (Freq >= 1000)
                    ? Freq / 1000 % 10
                    : 10;
    Seg_Buf[5] = (Freq >= 100)
                    ? Freq / 100 % 10
                    : 10;
    Seg_Buf[6] = (Freq >= 10)
                    ? Freq / 10 % 10
                    : 10;
    Seg_Buf[7] = Freq % 10;
    //高位熄灭2
    Seg_Buf[0] = (Freq / 10000000 % 10 == 0)
                    ? Freq / 10000000 % 10
                    : 10;
    Seg_Buf[1] = ((Freq / 1000000 % 10==0) && (Seg_Buf[0] == 10))
                    ? Freq / 1000000 % 10
                    : 10;
    Seg_Buf[2] = ((Freq / 100000 % 10==0) && (Seg_Buf[1] == 10))
                    ? Freq / 100000 % 10
                    : 10;
    Seg_Buf[3] = ((Freq / 10000 % 10==0) && (Seg_Buf[2] == 10))
                    ? Freq / 10000 % 10
                    : 10;
    Seg_Buf[4] = ((Freq / 1000 % 10==0) && (Seg_Buf[3] == 10))
                    ? Freq / 1000 % 10
                    : 10;
    Seg_Buf[5] = ((Freq / 100 % 10==0) && (Seg_Buf[4] == 10))
                    ? Freq / 100 % 10
                    : 10;
    Seg_Buf[6] = ((Freq / 10 % 10==0) && (Seg_Buf[5] == 10))
                    ? Freq / 10 % 10
                    : 10;
    Seg_Buf[7] = Freq % 10;
```

代码说明：

- 普通显示：直接显示各位数值
- 高位熄灭1：通过比较数值大小决定显示或熄灭
- 高位熄灭2：通过检测当前位是否为0且前一位是否熄灭来决定显示状态

数码管闪烁效果实现

数码管闪烁是一种常见的显示效果，其实现原理非常简单。主要通过以下两个关键变量来控制：

1. **位变量(bit)：** 用于控制数码管的显示状态（显示/熄灭）
2. **计时变量：** 用于控制闪烁的时间间隔

核心思路是在定时器中断中周期性地翻转位变量，从而实现数码管的交替显示和熄灭。

代码实现

```
// 闪烁控制变量
bit Seg_Show_Light = 0;         // 控制显示状态的位变量
unsigned int Time_500ms = 0;    // 500ms计时变量

void Seg_Proc(void)
{
    // 根据闪烁状态决定显示1还是熄灭
    Seg_Buf[0] = Seg_Show_Light ? 1 : 10;    // 闪烁显示1
}

void Timer1_Isr(void) interrupt 3
{
    uwTick++;    // 系统计时增加

    // 数码管位置循环更新
    Seg_Pos = (++Seg_Pos) % 8;

    // 数码管显示处理：自动判断是否需要显示小数点
    if (Seg_Buf[Seg_Pos] > 20)
        Seg_Disp(Seg_Pos, Seg_Buf[Seg_Pos] - ',', 1);    // 显示带小数点的数字
    else
        Seg_Disp(Seg_Pos, Seg_Buf[Seg_Pos], 0);          // 显示不带小数点的数字

    // 闪烁时间控制（500ms翻转一次）
    if (++Time_500ms == 500)
    {
        Time_500ms = 0;
        Seg_Show_Light ^= 1;    // 位变量取反
    }
}
```

实现要点：

1. `Seg_Show_Light` 使用bit类型声明，既节省内存又便于位操作
2. `Time_500ms` 用于500ms定时，每次中断累加，达到500后清零
3. 使用三目运算符（?:）简化显示逻辑判断

4. 位运算符（^=）实现状态翻转，比赋值取反更高效